

ОТЧЁТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №1

1. Создайте новый локальный репозиторий.

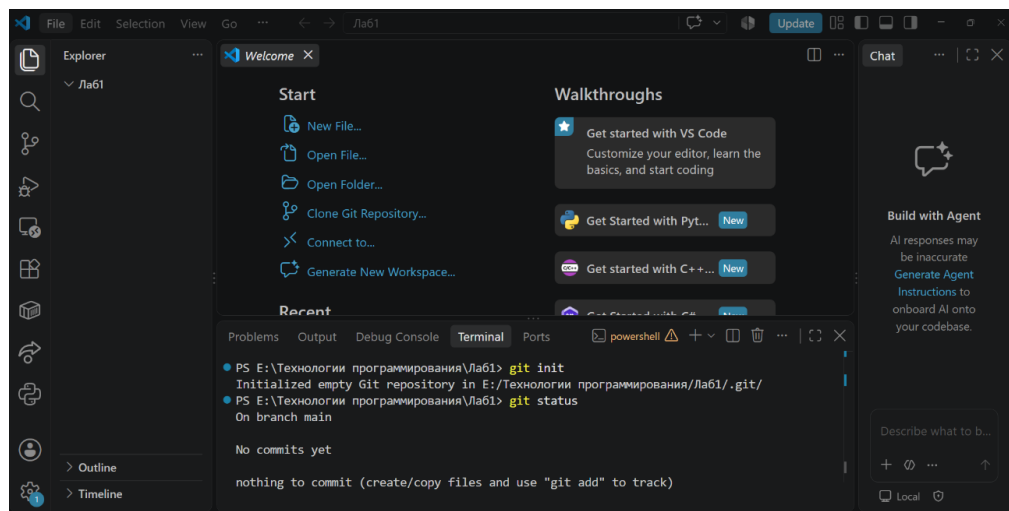


Рисунок 1 – инициализация репозитория в проекте, создание папки .git

2. Создайте два текстовых файла, добавьте их в репозиторий и сделайте первый коммит.

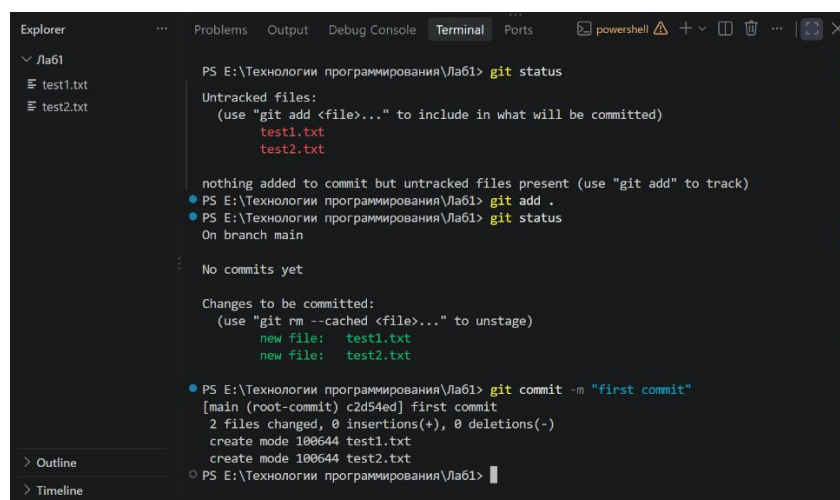
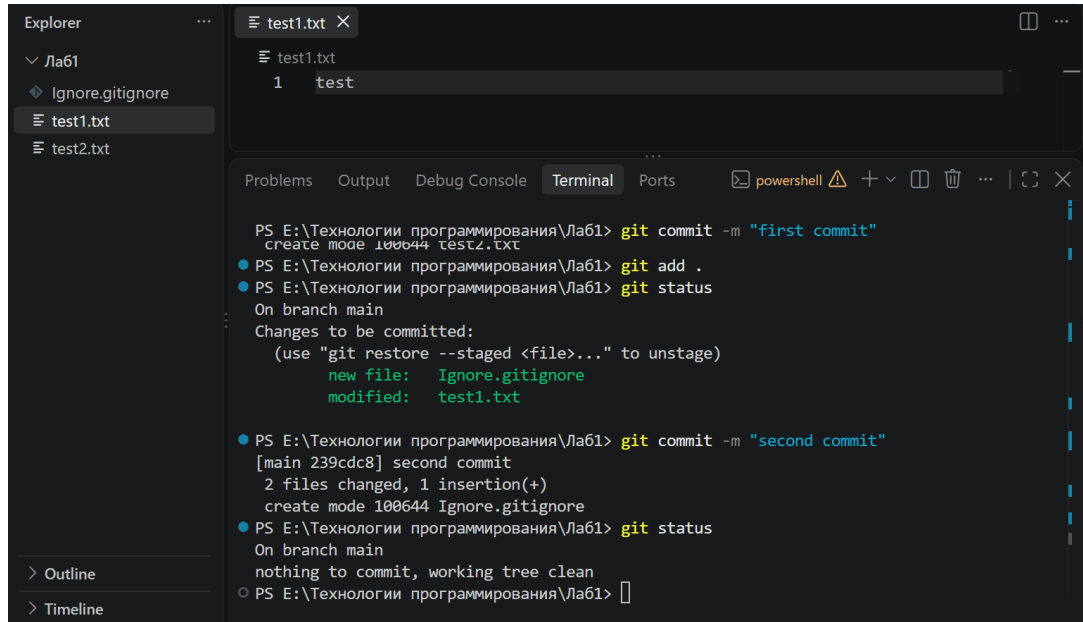


Рисунок 2 – Первый коммит. Созданы файлы test1.txt и test2.txt, добавлены в индекс (git add .) и зафиксированы коммитом "first commit".

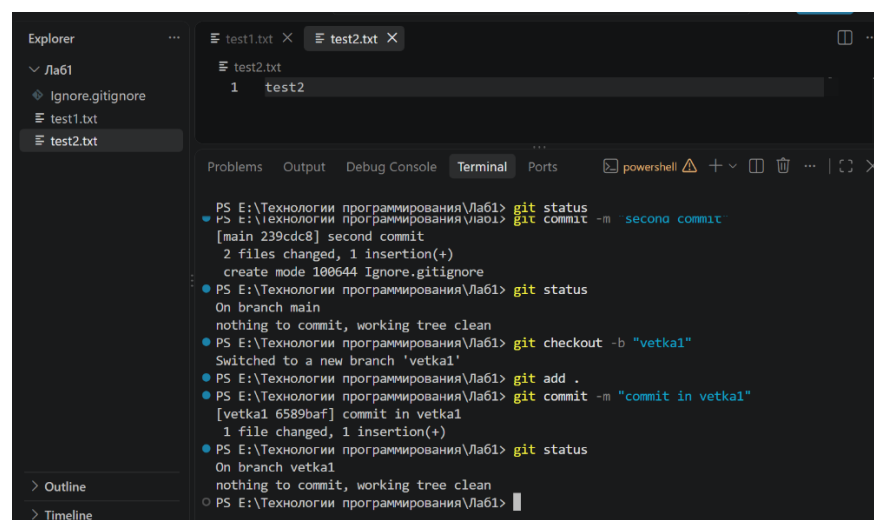
3. Добавьте файл .gitignore, в который включите шаблон для игнорирования определённых файлов.
4. Измените один из отслеживаемых файлов и сделайте второй коммит.



```
PS E:\Технологии программирования\Ла61> git commit -m "first commit"
create mode 100644 test2.txt
PS E:\Технологии программирования\Ла61> git add .
PS E:\Технологии программирования\Ла61> git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   Ignore.gitignore
        modified:   test1.txt
PS E:\Технологии программирования\Ла61> git commit -m "second commit"
[main 239cdc8] second commit
2 files changed, 1 insertion(+)
create mode 100644 Ignore.gitignore
PS E:\Технологии программирования\Ла61> git status
On branch main
nothing to commit, working tree clean
PS E:\Технологии программирования\Ла61>
```

Рисунок 3 – Второй коммит. Создан файл Ignore.gitignore, а также изменен файл test1.txt. Оба изменения добавлены в индекс и зафиксированы одним коммитом "second commit".

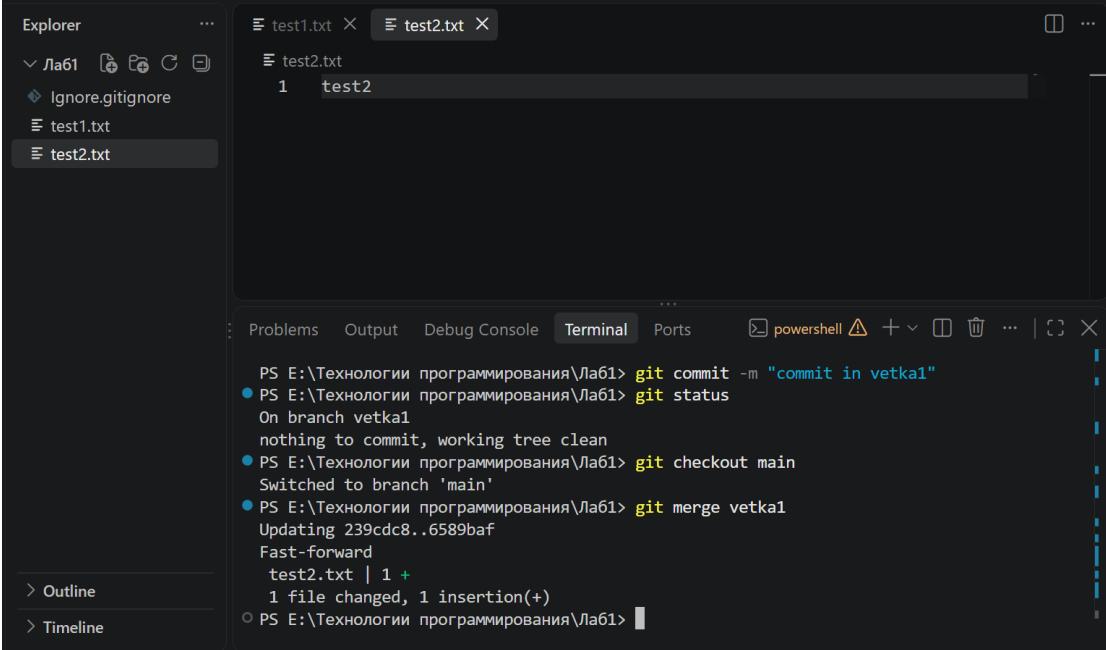
5. Создайте новую ветку для изменений. Переключитесь в неё, внесите изменения в другой файл и сделайте коммит.



```
PS E:\Технологии программирования\Ла61> git status
[main 239cdc8] second commit
2 files changed, 1 insertion(+)
create mode 100644 Ignore.gitignore
PS E:\Технологии программирования\Ла61> git status
On branch main
nothing to commit, working tree clean
PS E:\Технологии программирования\Ла61> git checkout -b "vetka1"
Switched to a new branch 'vetka1'
PS E:\Технологии программирования\Ла61> git add .
PS E:\Технологии программирования\Ла61> git commit -m "commit in vetka1"
[vetka1 6589baf] commit in vetka1
1 file changed, 1 insertion(+)
PS E:\Технологии программирования\Ла61> git status
On branch vetka1
nothing to commit, working tree clean
PS E:\Технологии программирования\Ла61>
```

Рисунок 4 – Создание ветки. Создана и активирована ветка vetka1 (git checkout -b vetka1), в файл test2.txt внесены изменения, выполнен коммит "commit in vetka1".

6. Вернитесь в основную ветку и выполните слияние с новой веткой. Слияние должно быть fast-forward (без конфликтов).

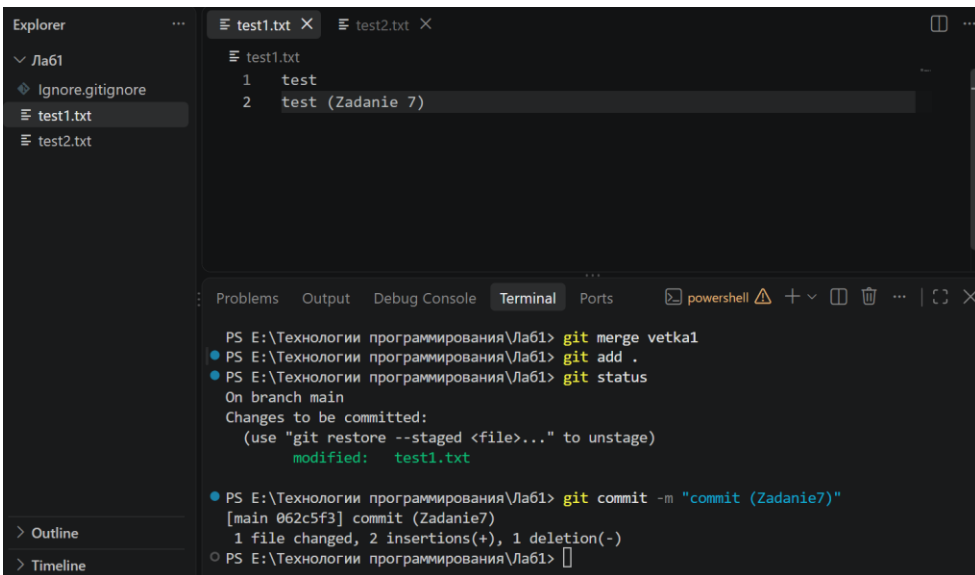


The screenshot shows the Visual Studio Code interface. The Explorer sidebar on the left shows a project named 'Ла61' with files 'test1.txt' and 'test2.txt'. The main editor displays 'test2.txt' with the content '1 test2'. The integrated terminal at the bottom shows the following commands and output:

```
PS E:\Технологии программирования\Ла61> git commit -m "commit in vetka1"
PS E:\Технологии программирования\Ла61> git status
On branch vetka1
nothing to commit, working tree clean
PS E:\Технологии программирования\Ла61> git checkout main
Switched to branch 'main'
PS E:\Технологии программирования\Ла61> git merge vetka1
Updating 239cdc8..6589baf
Fast-forward
 test2.txt | 1 +
 1 file changed, 1 insertion(+)
```

Рисунок 5 – Fast-forward слияние. Выполнен переход в ветку main (git checkout main) и слияние (git merge vetka1).

7. Сделайте ещё одно изменение в основной ветке и зафиксируйте его КОММИТОМ.



The screenshot shows the Visual Studio Code interface. The Explorer sidebar on the left shows a project named 'Ла61' with files 'test1.txt' and 'test2.txt'. The main editor displays 'test1.txt' with the content '1 test' and '2 test (Zadanie 7)'. The integrated terminal at the bottom shows the following commands and output:

```
PS E:\Технологии программирования\Ла61> git merge vetka1
PS E:\Технологии программирования\Ла61> git add .
PS E:\Технологии программирования\Ла61> git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   test1.txt
PS E:\Технологии программирования\Ла61> git commit -m "commit (Zadanie7)"
[main 062c5f3] commit (Zadanie7)
 1 file changed, 2 insertions(+), 1 deletion(-)
```

Рисунок 6 – Коммит в основной ветке. В файл test1.txt внесены изменения, выполнен коммит "commit (Zadanie7)".

8. Перейдите обратно в новую ветку и измените тот же файл, что и в основной ветке. Выполните коммит, а затем попробуйте слить ветки. Должен произойти конфликт. Разрешите его вручную и сделайте коммит.

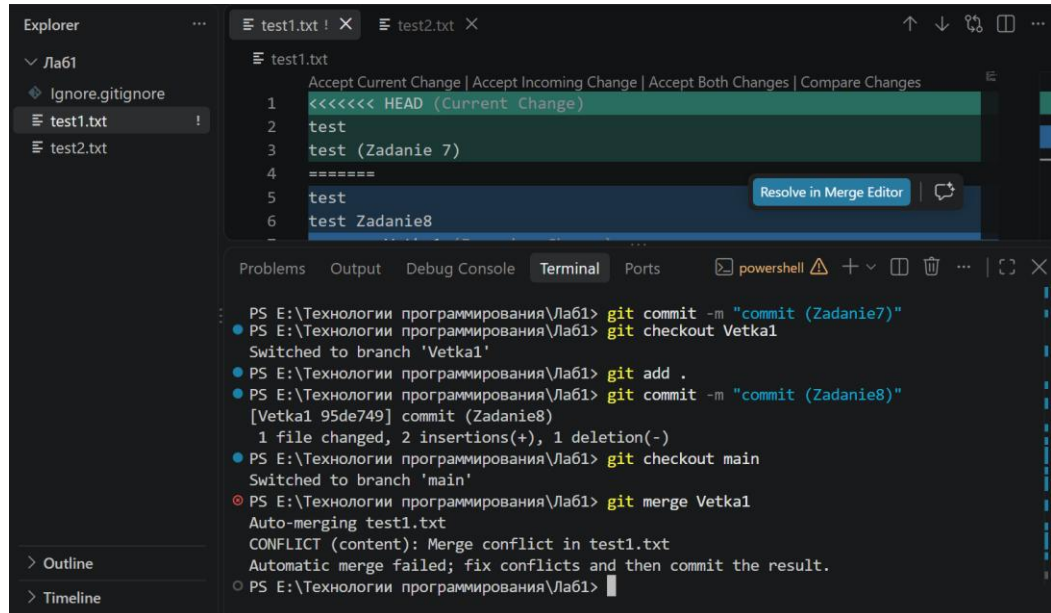


Рисунок 7 – В ветке main изменен файл test1.txt (коммит "commit (Zadanie7)"). При попытке слияния с веткой vetka1 возник конфликт.

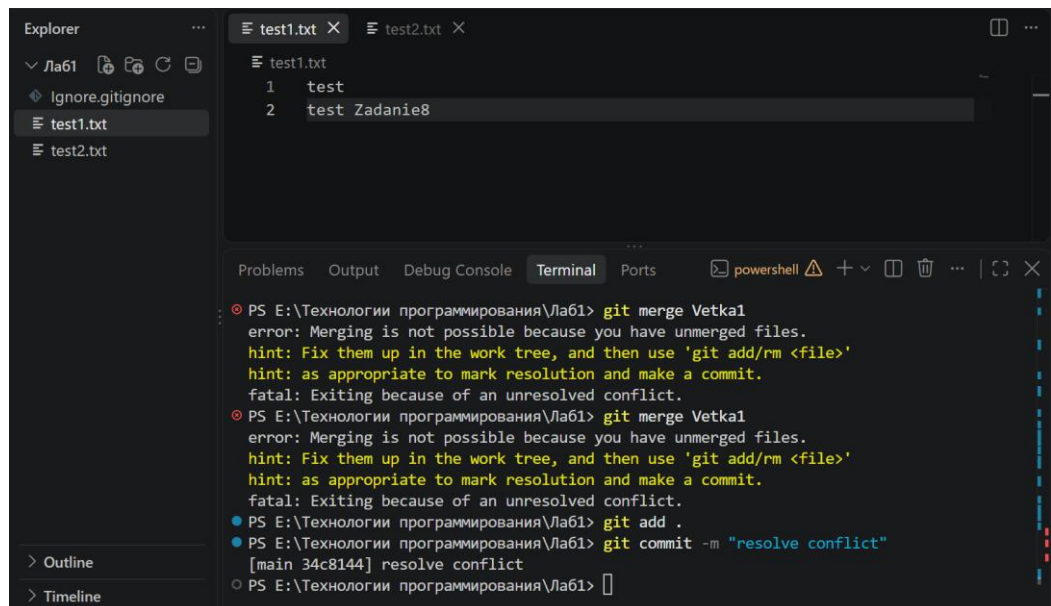
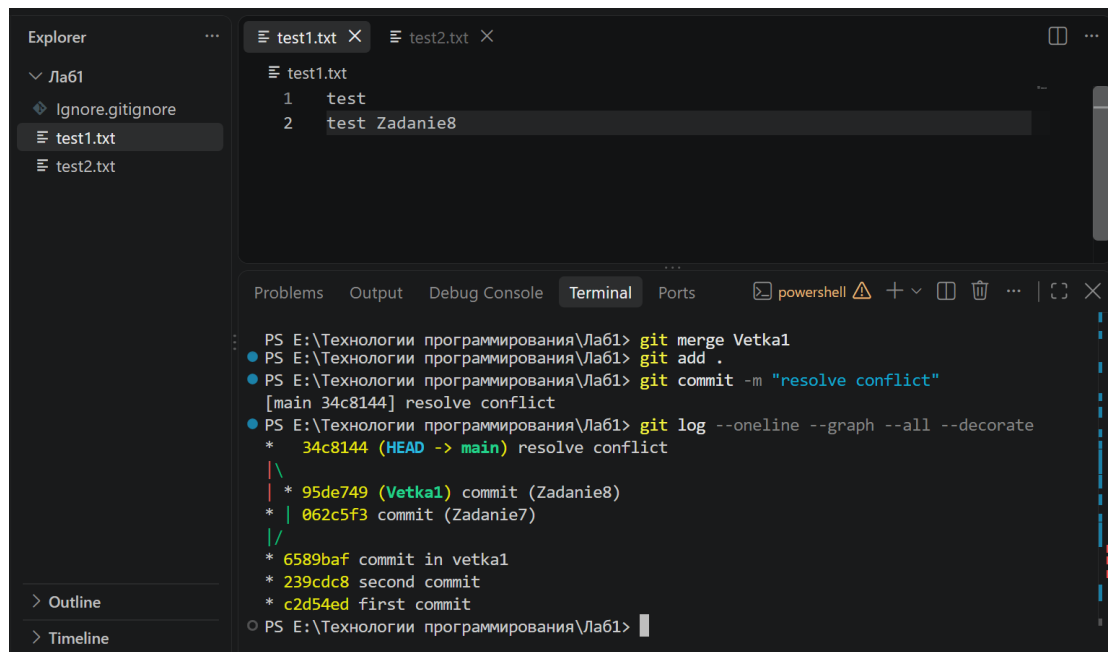


Рисунок 8 – Конфликт разрешен вручную, изменения зафиксированы коммитом "resolve conflict".

9. Просмотрите историю коммитов с отображением ветвлений.



```
test1.txt
1 test
2 test Zadanie8

PS E:\Технологии программирования\Лаб1> git merge Vetka1
PS E:\Технологии программирования\Лаб1> git add .
PS E:\Технологии программирования\Лаб1> git commit -m "resolve conflict"
[main 34c8144] resolve conflict
PS E:\Технологии программирования\Лаб1> git log --oneline --graph --all --decorate
* 34c8144 (HEAD -> main) resolve conflict
|
| * 95de749 (Vetka1) commit (Zadanie8)
| * | 062c5f3 commit (Zadanie7)
|/
* 6589baf commit in vetka1
* 239cdc8 second commit
* c2d54ed first commit
PS E:\Технологии программирования\Лаб1>
```

Рисунок 9 – Выполнена команда `git log --oneline --graph --all --decorate`, отображающая структуру ветвления.

10. Зарегистрируйтесь на GitHub (если ещё не зарегистрированы) и создайте удалённый репозиторий. Свяжите его с локальным и отправьте в него все КОММИТЫ.

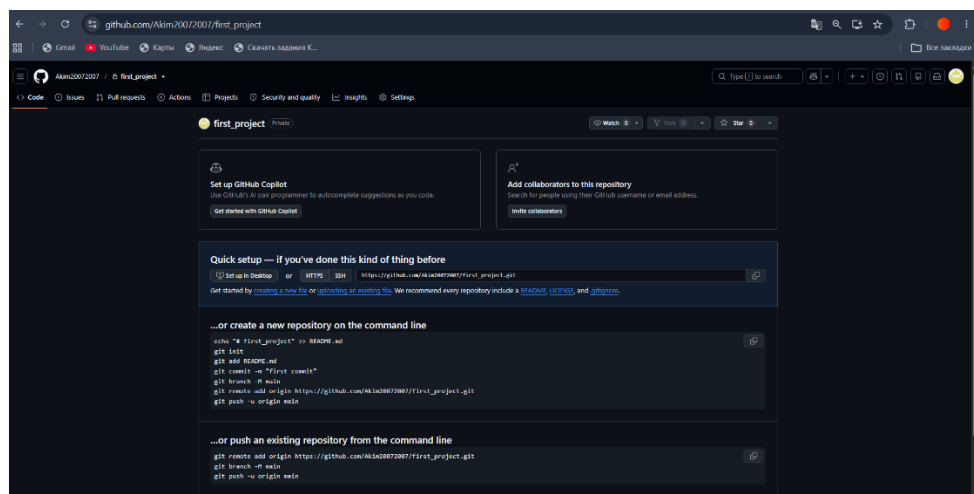


Рисунок 10 – Создание удалённого репозитория. На GitHub создан пустой репозиторий `first_project`.

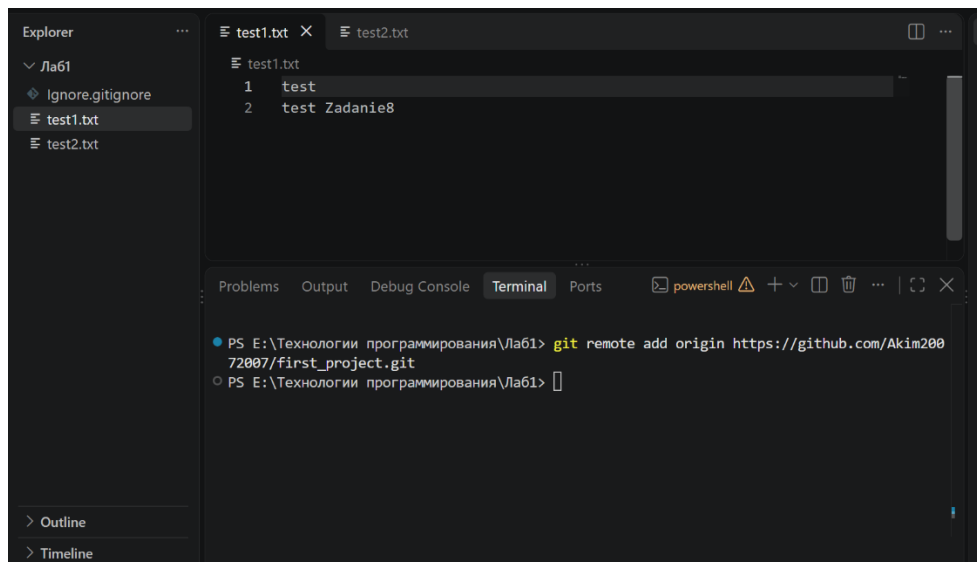


Рисунок 11 – Связывание с GitHub. Выполнена команда `git remote add origin ...` для связи локального репозитория с удалённым.

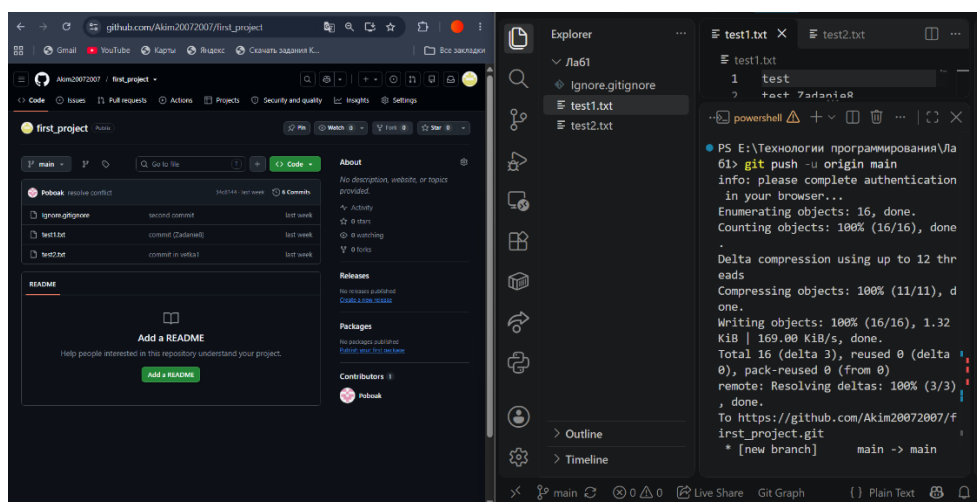


Рисунок 12 – Выполнена команда `git push -u origin main`, все локальные коммиты отправлены на GitHub.

11. На GitHub создайте файл README.md с кратким описанием проекта.

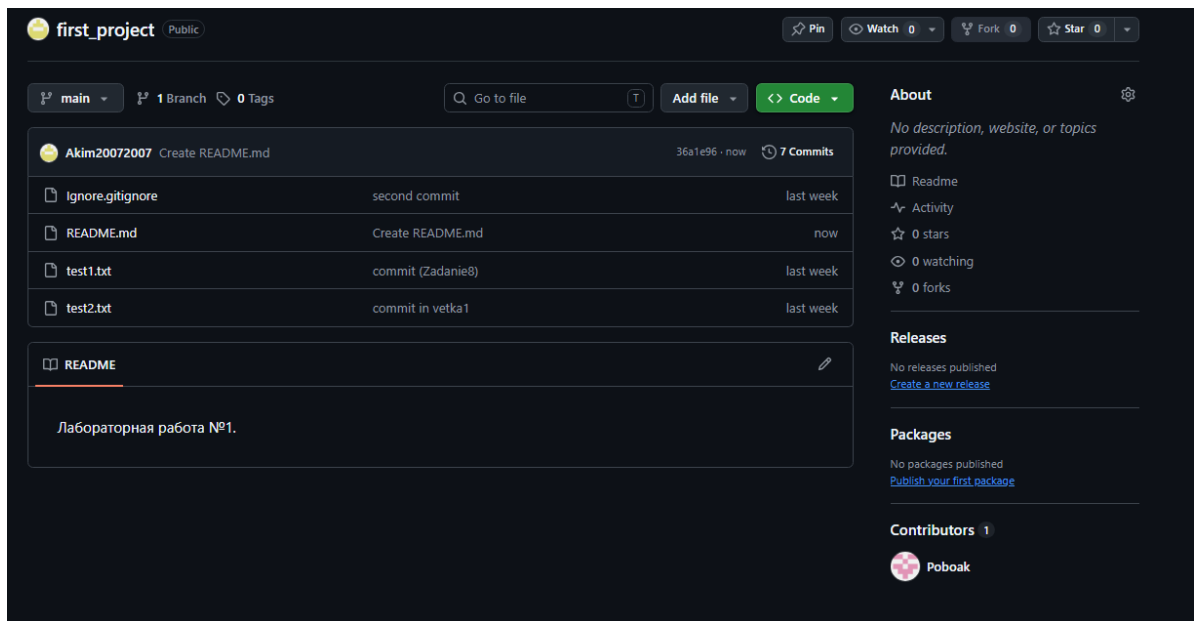


Рисунок 13 – Создание README.md на GitHub.

12. В локальном репозитории получите изменения с GitHub с помощью команды pull, чтобы скачать файл README.md.

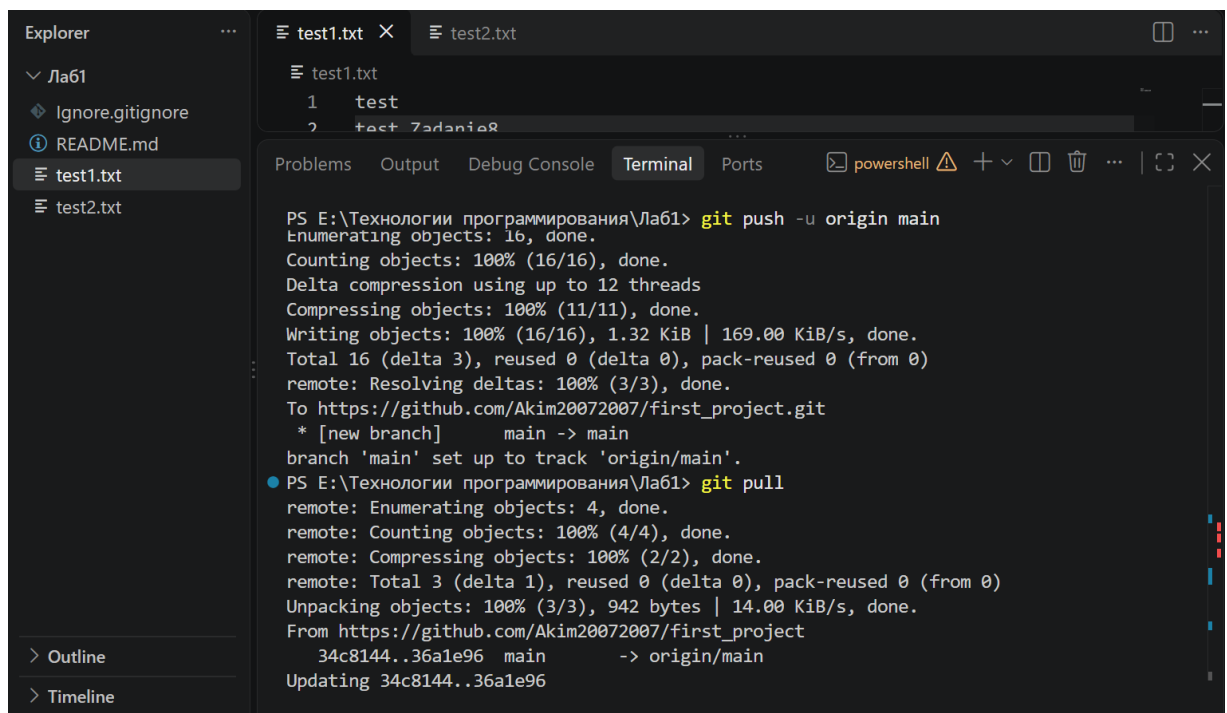


Рисунок 14 – Выполнена команда git pull, файл README.md скачан с GitHub в локальный репозиторий.